

CustomersDot main troubleshoot documentation

Overview

customers.gitlab.com is the site where GitLab customers can manage their subscription(s) for GitLab.com.

For all availability issues see the **escalation process for incidents or outages**.

Production and Staging

The production and staging environments reside in Google Cloud projects.

- Staging: gitlab-subscriptions-staging
- Production: gitlab-subscriptions-prod

SSH SSH connectivity to customer-dot VMs is established through Teleport. SREs don't require any additional teleport permissions to access these VMs, and can do so directly as follow:

```
$ tsh login --proxy=production.teleport.gitlab.net
$ tsh ssh customers-01-inf-prdsub
```

NOTE: if your user ID in Okta differs from that in Chef, you would need to make sure they match in order for this to work.

Break-glass procedure for SSH access SSH connectivity is normally established through Teleport. In the event that Teleport is unavailable, SREs can connect using SSH to the VMs through the Identity-Aware Proxy (IAP). For this to work, you need to file an MR to add the necessary firewall rule:

- Staging
- Production
- You might need to refer to these instructions

[!important] Ensure you revert the MR once you're back to using Teleport.

Once the firewall rule has been added, you should be able to do the following:

```
gcloud --project <PROJECT_ID> compute ssh <USERNAME>@<VM_NAME> --tunnel-through-iap
```

Example:

```
gcloud --project gitlab-subscriptions-staging compute ssh gservat@customers-01-inf-stgsub --
```

[!note] If you have trouble connecting via the IAP proxy, manually remove your SSH keys from the GCP instance metadata and try again. This may resolve the issue.

CDN Both staging and production services are proxied through Cloudflare. Refer to the section for NGINX below for rate limit information.

NGINX The web server on the VM has a rate limit set that should return a 429 when the rate is exceeded. This rate limit is specific to API Seat Requests and is managed in Ansible.

Logs

- Local Logs
 - NGINX Logs: `/var/log/nginx`
 - PostgreSQL Logs: `/var/log/postgresql`
 - Application Logs (Rails, Puma): `/home/customersdot/CustomersDot/current/log`
 - Sidekiq Logs: `/home/customersdot/CustomersDot/current/log`
- Stackdriver Logs
 - Application Logs from the VM are shipped into Stackdriver in GCP.
 - <https://cloudlogging.app.goo.gl/Jew7kUFaW8SUgeew9>

Metrics Specifications for a Customersdot metric catalog is available at `metrics-catalog/services/customersdot.jsonnet`.

This catalog references the following SLIs:

- `gitlab_sli_customers_dot_requests_total`
- `gitlab_sli_customers_dot_requests_error_total`
- `gitlab_sli_customers_dot_requests_error_apdex_total`
- `gitlab_sli_customers_dot_requests_error_apdex_success_total`
- `gitlab_sli_customers_dot_sidekiq_jobs_total`
- `gitlab_sli_customers_dot_sidekiq_jobs_error_total`
- `gitlab_sli_customers_dot_sidekiq_jobs_error_apdex_total`
- `gitlab_sli_customers_dot_sidekiq_jobs_error_apdex_success_total`

These SLIs were introduced as a Rails application SLI for CustomersDot (see the Collector class and the Metrics::Slis class for more details).

Two Prometheus instances have been set up for CustomersDot:

- Prometheus instance for Staging
- Prometheus instance for Production

Here is the main Grafana page for CustomersDot

Database and Rails console access Access to the CustomerDot DB and Rails console is via Teleport

SREs have SSH access to the CustomerDot VMs for emergency access to the database. Ensure all database changes outside of Teleport or application code are retroactively approved via a change request following the Infrastructure change request workflow

Once you have logged into the Staging or the Production VM, run one of these commands to open the database or the Rails console:

```
gitlab-db # opens the PSQL console.  
gitlab-rails-console # opens the Rails console.
```

These scripts can be found in `/usr/local/bin/`.

Database access with cloudsqlsuperuser permissions The Database Operations Team also has their own user `DBO_team` on CustomersDot DB Instances with cloudsqlsuperuser permissions. This user is necessary for maintenance operations and similar that require the highest possible level of access. It should not be used as a substitute for the teleport connection process, only for when the teleport process does not give sufficient access.

Ensure all database changes outside of Teleport or application code are approved or retroactively approved via a change request following the Infrastructure change request workflow

This user can access CustomerDot DB via the gcli cmd line as follows

```
gitlab-subscriptions-prod gcloud sql connect customers-db-2780  
--user=dbo_team --database=CustomersDot_production gitlab-subscriptions-  
staging gcloud sql connect customers-db-4dc5 --user=dbo_team  
--database=CustomersDot_stg gitlab-subscriptions-stg-ref gcloud sql  
connect customers-db-04f6 --user=dbo_team --database=CustomersDot_stg-ref
```

The passwords for this user are found in the Production repo of 1Password in the format `{gcp project} {instance name} {username}` for example `gitlab-subscriptions-staging customers-db-4dc5 dbo_team` This cmd will open a psql process in gcli to allow the team member to interact with the database.

Application provisioning The provisioning of the CustomersDot application stack is done through the CustomersDot Ansible project. For staging and production, provisioning works through Teleport to reach the VMs. For `stgsub-ref`, it is still direct SSH access through the bastion.

To provision CustomersDot in staging and production manually, please refer to this documentation.

Alerts related to provisioning are sent to the `#s_fulfillment_status` Slack channel.

Deployments When a pipeline is triggered on the `staging` (default) branch of CustomersDot, the application is first deployed to Staging then to Production after a delay of 2 hours.

That being said, it is possible to trigger a manual pipeline to deploy to production right away, should the need to do so arise. To do so, please refer to this documentation.

Alerts related to deployments are sent to the `#s_fulfillment_status` Slack channel.

If there's a need to restart services, please refer to this restart documentation.

Similar to application provisioning, deployments in staging and production environments work through Teleport (not direct SSH). For `stgsub-ref`, it is still direct SSH via bastion.

Change Management

Terraform is used to provision the virtual infrastructure for staging and production:

- Staging
- Production

Chef is used essentially to bootstrap user access for users and Ansible.

- Staging Base
- Production Base

Ansible is used to deploy the latest code to staging.

Alerting

At the moment, we rely on this Uptime Kuma instance for CustomersDot production for stack monitoring. When an Uptime Kuma alert is created from this instance, it is sent to the `#s_fulfillment_status` Slack channel.

Unschedule maintenance +When Zuora is down (reported to `#s_fulfillment_status`) and CustomersDot is in auto-maintenance mode for longer than 5 minutes, we should create an S3 incident with the CMOC updating the status page based on updates from <https://trust.zuora.com/>.

Service degradation during SeatLink traffic hour `maintenance_mode_seat_link` is a feature flag for just blocking seat link traffic. This was implemented because we suspect SeatLink was the cause of several service degradations. If a similar degradation occurs during SeatLink traffic hour (typically around 4 AM UTC), we can enable this flag to block SeatLink traffic.

Customer Impacts of Outage

Gitlab.com customers For Gitlab.com customers there are several ways customers interact with the Customers Portal:

- When they click Buy `storage` or Buy `CI minutes` buttons on the Usage Quotas page. Both buttons redirect the customer to the Customers Portal. This will not be available during downtime.

- When they attempt to validate themselves by entering their credit card information when a CI pipeline is blocked by the anti-abuse system. This will not be available during downtime.

Self-managed and Dedicated Customers

- When a customer purchases a license for a self-managed or dedicated instance, they receive an activation code, which they enter on their instance to activate the license. For the activation to work, the instance needs to communicate with the Customer Portal. You can read more about how that works. This will not be available during downtime.
- If a customer does try to activate a subscription during downtime it will fail with the `Cannot activate instance due to a connectivity issue` error message.
- Customers will not be able to obtain Cloud Connector access credentials anymore, which are needed to enable Duo and other features. Failure to sync with the Customers Portal for longer than the TTL of these credentials means these features will become unavailable.